

✓ 100 XP 

Configure AI/BI Genie instructions

10 minutes

Making your data discoverable involves more than organizing it into catalogs and schemas. **AI/BI Genie** in Azure Databricks enables business users to explore data through **natural language questions**, but it needs guidance to interpret your organization's terminology and business logic. When you configure AI/BI Genie instructions properly, you transform Unity Catalog metadata into a **knowledge store** that helps both humans and AI systems understand your data.

Microsoft Azure

 databricks



databricks-learn





My Genie Space

+ New chat



 Configure

 Benchmarks

 Monitoring

 Share



how many products do I have in my catalog?

Analysis (click to view)  dim_product

You have a total of **606 products** in your catalog.

1 row  [Show code](#) 

	1.2 product_count
1	606

Was this correct?   

what is the average price?

Analysis (click to view)  dim_product

The **average price** of products in your catalog is **747.66**.

1 row  [Show code](#) 

	1.2 average_price
1	747.66

 Ask your question... 

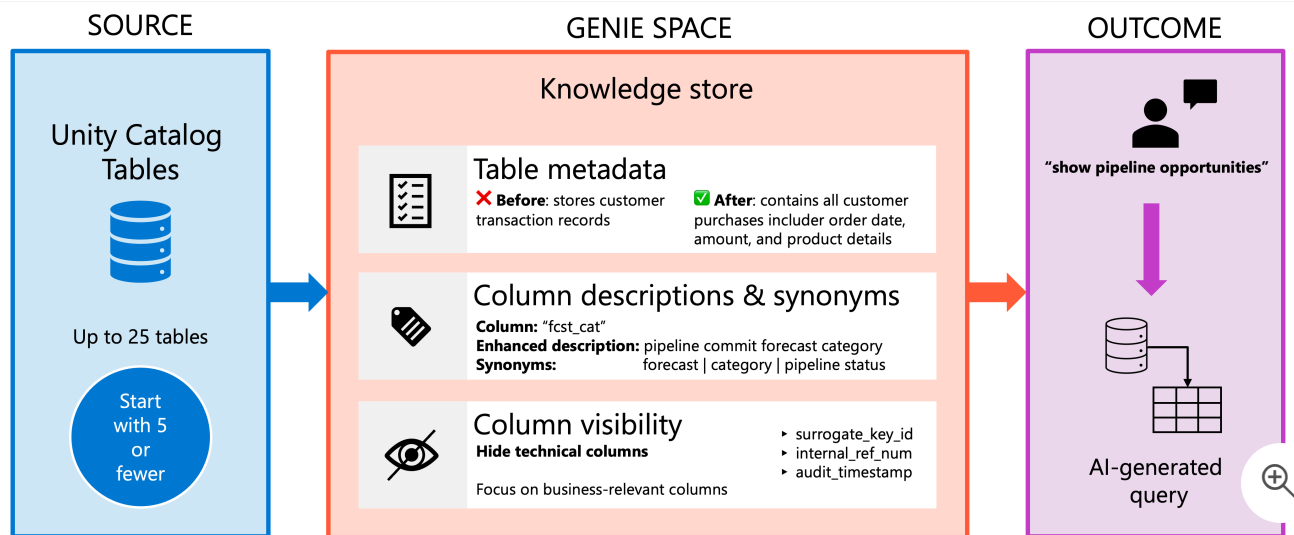
Always review the accuracy of responses. 

Configure the knowledge store

The **knowledge store** is your primary tool for teaching AI/BI Genie about your data. You can customize **metadata** at the **space level** without altering the underlying Unity Catalog objects. This gives you flexibility to refine descriptions and add context specific to how business users interact with the data.

<https://learn.microsoft.com/en-us/training/modules/create-and-organize-objects-in-unity-catalog/9-configure-ai-bi-genie-instructions>

2/7



When you create a Genie space, you start by selecting up to **25 tables or views** from Unity Catalog. These data objects form the foundation of your space. The key is choosing tables that answer a focused set of business questions rather than including everything available. A sales manager might need access to opportunity and account tables, while a logistics manager requires shipment and inventory data. Keep your initial selection small. **Five tables or fewer** works well. Expand only as users request additional data.

After adding tables, you enhance their discoverability by editing metadata. Select **Configure > Data** in your Genie space, then select a table to view its columns. Each table includes a **description field** where you can explain its purpose using **business language**. Instead of technical descriptions like "stores customer transaction records," write "contains all customer purchases including order date, amount, and product details." This context helps Genie generate accurate SQL when users ask questions about purchases or orders.

Column-level metadata requires similar attention. Unity Catalog preserves column names and descriptions from the source tables, but you can refine them within the Genie space. Suppose you have a column named `fcst_cat` that stores forecast categories. In the space, you update its description to "indicates whether an opportunity is in the Pipeline, Best Case, or Commit forecast category" and add **synonyms** like "forecast," "category," and "pipeline status." When users ask about pipeline opportunities, Genie matches their language to your column using these synonyms.

You can also **hide columns** that don't serve business users. Technical columns like surrogate keys, internal identifiers, or audit timestamps often confuse both users and AI. Select the columns you

want to hide and choose **Hide selected columns** from the **Actions** menu. This keeps Genie focused on columns that matter for business analysis.

Define data relationships and prompt matching

With metadata in place, you help Genie understand how your tables connect and what values they contain. These configurations allow Genie to write accurate **JOIN statements** and match user questions to specific data values.

Unity Catalog can store **foreign key relationships** between tables, and Genie uses these to construct **JOIN** statements automatically. If your tables don't have foreign keys defined in Unity Catalog, or if you need more complex join logic, define relationships in the knowledge store. Navigate to the **Joins** section and specify how tables relate to each other.

For a sales pipeline dataset, you define a join between an `opportunity` table and an `accounts` table. Select both tables, enter the join condition `opportunity.accountid = accounts.id`, and specify the relationship type as **Many to one** because multiple opportunities can belong to a single account. This teaches Genie how to combine these tables when users ask questions like "show me opportunities by account region."

Prompt matching complements relationship definitions by giving Genie visibility into your actual data. When enabled, Genie collects representative values from columns, helping it match user prompts to real values in your tables. If a user asks about "opportunities in California" but your data uses state codes like "CA," prompt matching allows Genie to make that connection. Two components control how matching works: **format assistance** provides representative values for all eligible columns so Genie understands data types and formatting patterns, while **entity matching** curates distinct value lists for categorical columns. The system automatically enables prompt matching when you add tables, but you can manage which columns participate.

Entity matching is most useful for categorical columns where users are likely to reference specific entries—such as states, product categories, and region names. Genie can store up to **1,024 distinct values** per column across **120 columns** in your space. To enable entity matching for a column, select the pencil icon next to the column name, expand **Advanced settings**, and turn on **Entity matching**. As your data changes over time, refresh prompt matching data to keep the knowledge store current.

Add SQL instructions and examples

Beyond metadata and relationships, you teach Genie how to answer business questions by providing **SQL instructions**. Two types of instructions serve different purposes: **SQL expressions** define reusable business concepts, while **example SQL queries** show how to handle specific question patterns.

SQL expressions work best for common business terms that appear across many questions. Navigate to **Configure > Instructions > SQL Expressions** to define measures, filters, and dimensions. A measure might calculate gross margin as $(\text{revenue} - \text{cost}) / \text{revenue}$, while a dimension could extract the fiscal quarter from a date field. When you define these expressions with clear names and synonyms, Genie can incorporate them into generated queries without you writing complete SQL examples.

Example SQL queries address more complex scenarios where users ask multi-part questions or use domain-specific phrasing. These queries appear in the **SQL Queries** tab under **Instructions**. For each example, you write both the question users might ask and the SQL that answers it. If users frequently ask "What's our current pipeline by region?", provide that exact phrasing along with SQL that joins opportunity and account tables, filters for open opportunities in the pipeline forecast category, and groups results by region.

SQL

```
-- What's our current pipeline by region?
SELECT
    accounts.region AS Region,
    SUM(opportunity.amount) AS Pipeline
FROM opportunity
JOIN accounts ON opportunity.accountid = accounts.id
WHERE
    opportunity.forecastcategory = 'Pipeline'
    AND opportunity.stagename NOT LIKE '%closed%'
GROUP BY accounts.region;
```

Genie can either use example queries directly or learn from them to answer similar questions. When you add **parameters** to example queries using the syntax `:parameter_name`, Genie substitutes user-provided values at runtime. Parameterized example queries that match the user's exact question are marked as **Trusted** to give users confidence in the results. Non-parameterized example queries are used as context to guide Genie in generating similar SQL, but aren't marked as Trusted.

The **Text** tab holds general instructions that apply globally across all questions. Use this sparingly for context that doesn't fit into other formats. Examples include explaining that your fiscal year starts in February or that all timestamps should be converted to a specific timezone. Too many general instructions can dilute their effectiveness, so focus on essential business rules that Genie can't infer from your data.

Genie spaces enforce two separate limits. The **instructions limit** caps at **100** and covers example SQL queries (each counts as one), SQL functions (each counts as one), and the entire general text instructions block (counts as one). Column descriptions and prompt matching settings don't count toward this limit. The **knowledge store snippets limit** caps at **200** and covers table descriptions, join relationships, and SQL expressions (measures, filters, and dimensions) combined. Text instructions, example SQL queries, SQL functions, column descriptions, and prompt matching settings do not count toward the knowledge store snippets limit. These limits encourage you to prioritize quality over quantity.

Apply iterative refinement

Configuring AI/BI Genie is an **ongoing process**, not a one-time setup. Start with minimal instructions and expand based on actual user questions and feedback.



After configuring your initial space, test it yourself by asking questions you expect business users to pose. Review the generated SQL carefully to identify where Genie misinterprets data or business terminology.

When you find incorrect responses, use the **Show generated code** option to examine the SQL, edit it as needed, and save it as a new instruction by clicking **Add as instruction**. This teaches Genie how to handle similar questions in the future. You can also create **benchmark questions** that test Genie's accuracy systematically. Add common questions with their expected SQL answers, then run benchmarks regularly as you refine the space to measure improvement.

Encourage business users to provide feedback through the built-in response rating mechanism. When users mark responses as correct or incorrect, you can review their feedback in the **Monitoring** tab and adjust instructions accordingly. This collaborative refinement ensures your Genie space evolves to meet real business needs rather than assumptions about how users might ask questions.

Genie also learns on its own through **knowledge mining recommendations**. When you first create a space, Genie analyzes the Unity Catalog metadata for your selected tables and automatically saves any primary key and foreign key relationships it finds as join relationships in the knowledge store — so well-annotated schemas reduce manual setup from the start. Beyond initial discovery, whenever you approve a response (by marking it correct or downloading the results), Genie analyzes the underlying SQL and may suggest new SQL expressions — measures, filters, or dimensions — as well as additional join relationships to add to the knowledge store. Suggestions appear for your review and are not added automatically, keeping you in control of what guidance Genie relies on.

As you iterate, watch for patterns in failed queries. If Genie consistently uses the wrong table or column, review your metadata and descriptions. If it can't match user language to data values, check that prompt matching is enabled for relevant columns. If joins are incorrect, verify your relationship definitions. Each refinement makes the knowledge store more accurate and the space more valuable to users.

Next unit: Exercise - Create and Organize Objects in Unity Catalog

[< Previous](#)[Next >](#)